[Khanh Pham] - Socket Programming - TCP/UDP Sockets & I/O Multiplexing

Socket programming

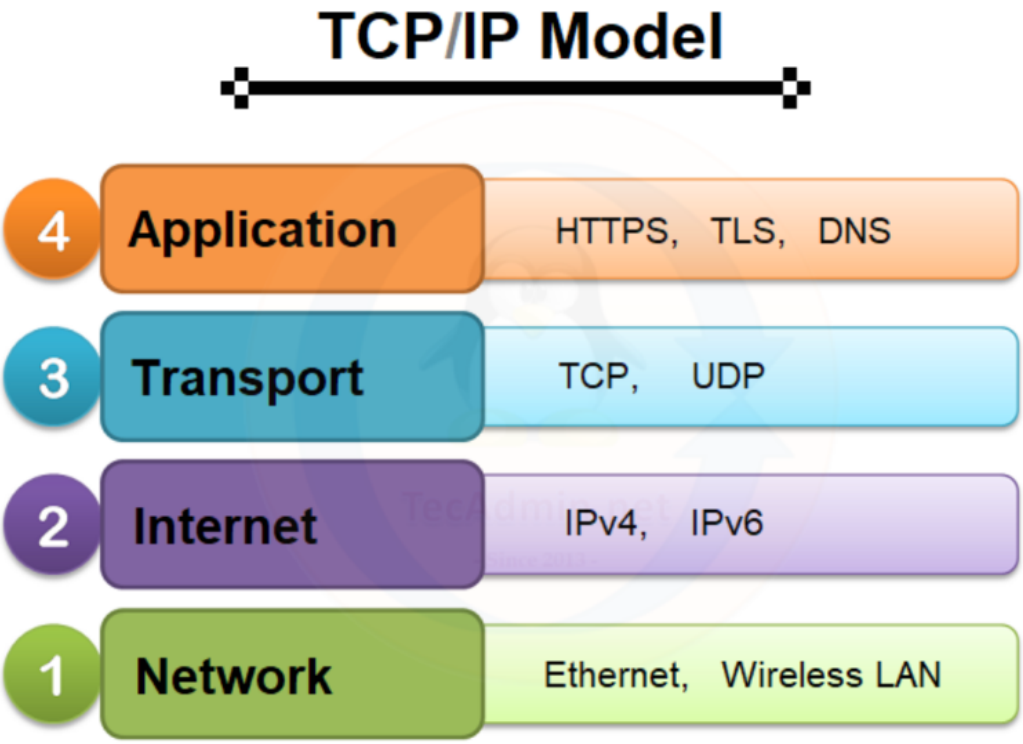
1. Socket instruction

In the previous part, we understand the way that processes can communicate together. However, this methods can only use for processes between 1 device. Therefore, we need a way to communicate between processes from 2 different devices.

To communicate between 2 different device, we need network. To use network, we need firstly know the OSI model(in this document, I will use TCP/IP model instead):

1.1. TCP/IP model

TCP/IP show all protocol each process need to communicate. We can see overview of this model by image bellow:



> I will write more about this model and it's element in the network document later. To understand this document, we just need know processes need transport layer(TCP, UDP) as a standard for communicating. Besides, they also need IPv4 or IPv6 as a address of each device.

1.2. BSD

As a other documents, there are a lot of api function we need to study, so before going to this api, I will write about the "father" of all api in socket programming: `BSD`

`BSD` is the opensource OS, it's build by an group in California University.

`BSD socket` is the standard of all socket apis, it is introduced by BSD in 1983.

> Almost other OS like Window, Linux, IOS, Androin follow BSD socket standard.

1.3. Socket address structure

Most of socket api need pass a socket address structure as an argument inside. I will list all type of socket structure before writing about more reason of implementing this structure.

IPv4 Socket Address structure

```
struct in_addr
{
    in_addr_t s_addr;           /* 32 bits IPv4 address */
};
struct sockaddr_in
{
    sa_family_t sin_family;     /* AF_INET */
    in_port_t sin_port;         /* 16-bit TCP or UDP port number */
    struct in_addr sin_addr;    /* 32-bit IPv4 address */
    char sin_zero[8];           /* unused */
};
```

> Total size of this structure is 16 bytes

Generic Socket Address Structure

```
struct sockaddr
{
    sa_family_t sa_family;      /* Address family */
    char sa_data[14];             /* Socket address */
};
```

> Total size of this structure is 16 bytes

IPv6 Socket Address Structure

```
struct in6_addr {
    uint8_t s6_addr[16];   /* 128-bit IPv6 address */
                           /* network byte ordered */
};

struct sockaddr_in6 {
    sa_family_t     sin6_family;    /* AF_INET6 */
    in_port_t       sin6_port;      /* transport layer port# */
                                    /* network byte ordered */
```

```
    uint32_t        sin6_flowinfo;  /* flow information, undefined */
    struct in6_addr sin6_addr;      /* IPv6 address */
                                    /* network byte ordered */
    uint32_t        sin6_scope_id;  /* set of interfaces for a scope */
};
```

> Total size of this structure is 28 bytes

> Now we have a question: Why we need the element `sin_zero` in IPv4 Socket address structure?

> > Firstly, according to the interface of some socket api function like `sendto()`, `bind()`, `connect()`,... ,they need the structure of generic address structure(sockaddr) instead of using specific structure. Therefore, we must explicitly cast `sockaddr_in`, or whatever address we using, to `sockaddr`. When kernel read 2 first bytes from this structure pointer, it will decide the next (IPv4 or IPv6) action.

```
ssize_t sendto(int socket, const void *message, size_t length,
               int flags, const struct sockaddr *dest_addr,
               socklen_t dest_len);
```

> > Secondly, we need 8 padding bytes of IPv4 socket address structure because the Buffer Over-read problem. As we know, kernel will decide the next action when receive the type of socket address structure. However, data isn't inside kernel space, so it need copy from userspace. Because the size of standard struture(sockaddr) is 16 bytes, the kernel will copy 16 bytes from userspace. If we don't have the sin_zero element, this is undefined behavior if we copy other memory

> from userspace.

1.4. Byte ordering and Manipulation

Most of developers use the device with litte-edian architechture. However, the embedded device use big-edian architechture. Therefore, if we want send data to another architechture device, we need exchange.

Some functions can help us:

```
uint16_t htons(uint16_t host16bitvalue); /* host to network */
uint32_t htonl(uint32_t host32bitvalue);

uint16_t ntohs(uint16_t net16bitvalue);  /* network to host */
uint32_t ntohl(uint32_t net32bitvalue);
```

For example:

```
int main()
{
    u_int16_t temp = 0x1234;
    printf("litte-edian: 0x%x\n",temp);
    printf("big-edian: 0x%x\n", htons(temp));
}
```

The result is:

```
litte-edian: 0x1234
big-edian: 0x3412
```

Some other functions:

```
void *memset(void *dest, int c, size_t len);
void *memcpy(void *dest, const void *src, size_t nbytes);
int memcmp(const void *ptr1, const void *ptr2, size_t nbytes);
```

1.5. Address function

We can convert address between ASCII strings (what humans prefer to use) and network byte ordered binary values (values that are stored in socket address structures) by some function

```
int inet_aton(const char *strptr, struct in_addr *addrptr);
in_addr_t inet_addr(const char *strptr);
char *inet_ntoa(struct in_addr inaddr);
int inet_pton(int family, const char *strptr, void *addrptr)
const char *inet_ntop(int family, const void *addrptr, char *strptr, size_t len);
```

2. TCP socket programming

As we know, before tranfering data in TCP protocol, we need set up a connection between 2 processes. To create this connection, we need some function: `socket()`, `bind()`, `listen()`, `connect()` and `accept()`. After that, when clients want to disconnect, they need the function: `close()`

2.1. TCP connection

This is all api of this functions:

- `socket()` creates an endpoint for communication and returns a file descriptor that refers to that endpoint

```
int socket(int family, int type, int protocol);
```

- `bind()` assigns the address specified by `addr` to the socket referred to by the file descriptor `sockfd`.

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- `listen()` marks the socket referred to by `sockfd` as a passive socket, that is, as a socket that will be used to accept incoming connection requests using `accept()`

```
int listen(int sockfd, int backlog);
```

- The `connect()` system call connects the socket referred to by the file descriptor `sockfd` to the address specified by `addr`

```
int connect(int sockfd, const struct sockaddr *servaddr, socklen_t addrlen);
```

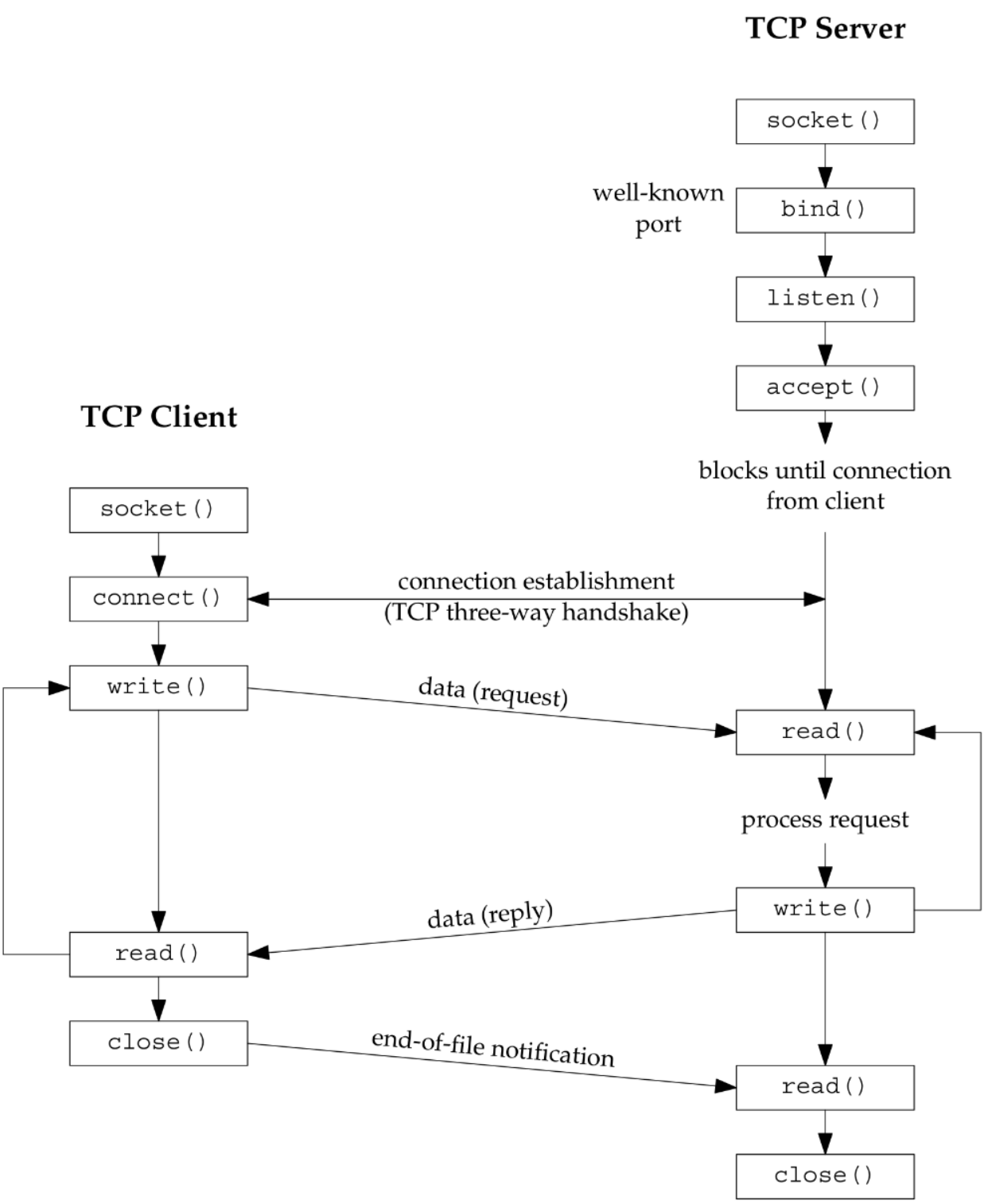
- `accept()` is called by a TCP server to return the next completed connection from the front of the completed connection queue

```
int accept(int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen);
```

- `close()` closes the socket descriptor.

```
int close(int fd);
```

To imagine all step when we want to create connection between 2 processes: client and server, we can see this image:



Some important information about this functions:

- We can use the `INADDR_ANY` of the element address or `0` in port of struct to make kernel will automatically choose address IP and port
- There are 2 types of queue: `incomplete connection queue` and `conplete connection queue` (before and after accepting `SYN`).
- If the queues are full, TCP will ignore the arriving `SYN`
- The accept function will make process suspended until receiving a suitable `SYN`

2.2. Concurrent Servers

In the situation when we have 1 server and more client, we can use multithread or multiprocess for server. That mean we can create other threads or child processes whenever accept a new connection.

> However, we should use the multi-process because threads share data together, so if we don't be careful when using mutual exclution, race condition or deadlock can be occured.

Now we can write an example of 1 client-1 server(chat program peer-to-peer) using socket programming. I also use 2 devices, which client IP address is `10.0.0.77`, server IP address is `10.0.0.81`

```
/* client from device 10.0.0.77*/

#define ADDRESS "10.0.0.81"
#define PORT 1234

int sockfd;

void *thread1()
{
    while (1)
    {
        char result[50];
        memset(&result, 0, sizeof(result));
        int bytes = read(sockfd, result, sizeof(result) - 1);
        if (bytes > 0)
        {
            result[bytes] = '\0';
        }
        printf("server to client: %s", result);
    }
}

int main()
{
    pthread_t pid;
```

```
    struct sockaddr_in seraddr;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    seraddr.sin_family = AF_INET;
    seraddr.sin_port = htons(PORT);
    inet_pton(AF_INET, ADDRESS, &seraddr.sin_addr);
    memset(&seraddr.sin_zero, 0, sizeof(seraddr.sin_zero));

    int result = connect(sockfd, (struct sockaddr *)&seraddr, sizeof(seraddr));

    printf("connection result = %d\n", result);
    pthread_create(&pid, NULL, thread1, NULL);

    while (1)
    {
        char temp[50];
        fgets(temp, sizeof(temp), stdin);
        write(sockfd, temp, sizeof(temp));
    }
}


/* server  from device 10.0.0.81*/

#define ADDRESS "10.0.0.81"
#define PORT 1234

int clientfd;

void *thread1()
{
    while (1)
    {
        char result[50];
        memset(&result, 0, sizeof(result));
        int bytes = read(clientfd, result, sizeof(result) - 1);
        if (bytes > 0)
        {
            result[bytes] = '\0';
```

```
        }
        printf("client to server: %s", result);
    }
}

int main()
{
    pthread_t pid;

    int sockfd = socket(AF_INET, SOCK_STREAM, 0);

    struct sockaddr_in seraddr;
    seraddr.sin_family = AF_INET;
    seraddr.sin_port = htons(PORT);
    inet_pton(AF_INET, ADDRESS, &seraddr.sin_addr);
    memset(&seraddr.sin_zero, 0, sizeof(seraddr.sin_zero));

    bind(sockfd, (struct sockaddr *)&seraddr, sizeof(seraddr));

    listen(sockfd, 5);

    struct sockaddr_in clientaddr;
    socklen_t clientlen;
    clientlen = sizeof(clientaddr);

    clientfd = accept(sockfd, (struct sockaddr *)&clientaddr, &clientlen);
    char client_address[20];
    inet_ntop(AF_INET, &clientaddr.sin_addr, client_address, sizeof(client_address));
    int port = ntohs(clientaddr.sin_port);
    printf("client ip address: %s\nclient port %d\n", client_address, port);
    pthread_create(&pid, NULL, thread1, NULL);

    while (1)
    {
        char temp[50];
        fgets(temp, sizeof(temp), stdin);
        write(clientfd, temp, sizeof(temp));
    }
}
```

The result is:

```
/* Client */
connection result = 0
hello
how are you
server to client: I fine, thank you, and you

/* server */
client ip address: 10.0.0.77
client port 51170
client to server: hello

client to server: how are you

I fine, thank you, and you
```

2.3. Getsockname and getpeername function

As we know from the previous part, because clients don't need `bind()` function to assign address and port, so they will not know their IP local address and port. We can call `getsockname()` to know this:

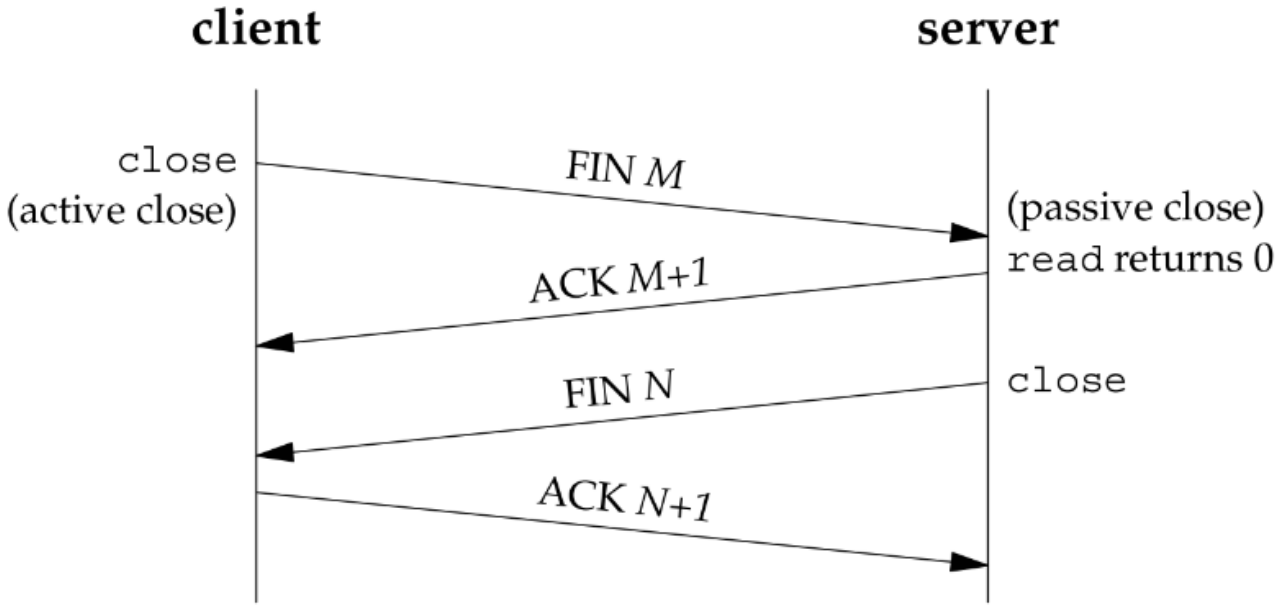
```
int getsockname(int sockfd, struct sockaddr *localaddr, socklen_t *addrlen);
```

Besides, we know the standard pattern of concurrent servers is server call exec after forking, so in this time, the forking server will not remember the client IP address and port, so they need a function to read it. `getpeername()` function will do that.

```
int getpeername(int sockfd, struct sockaddr *peeraddr, socklen_t *addrlen);
```

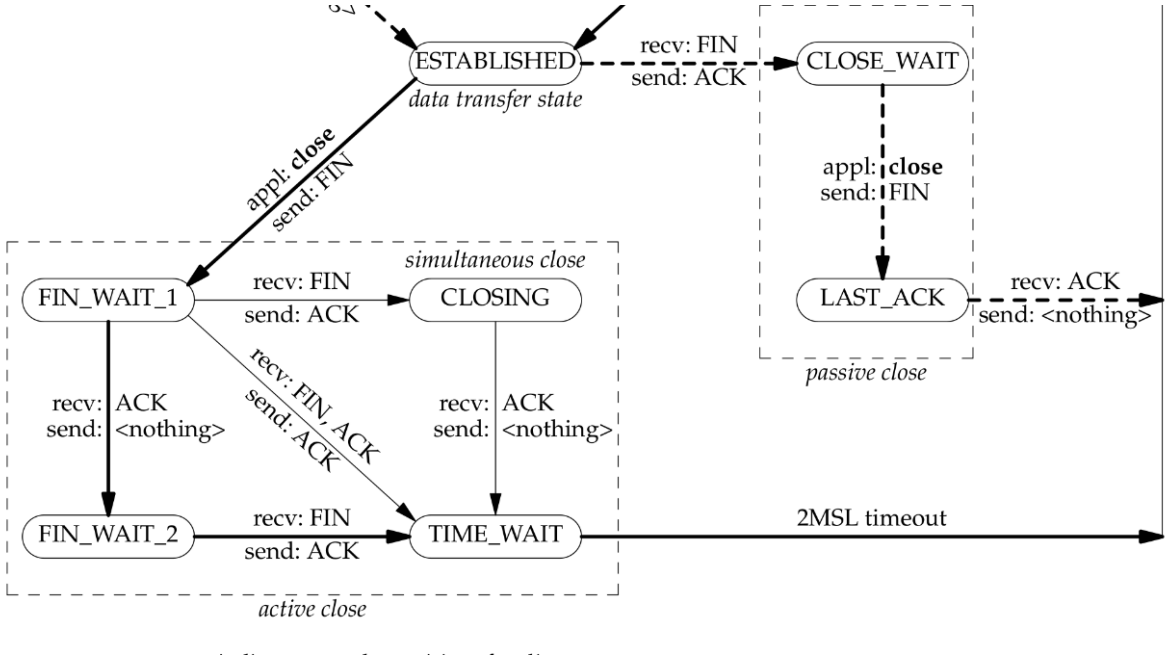
2.4. The time-wait state

As we know, when the tcp socket connection need to disconnect with other sockets, they will do `4-ways-handshake`.



> However, we know TCP is the reliable connection, so we have a situation in `active close` side: The final ack can be lost. So we need the way to handle this problem: **Time-wait state**

To understand the state of connection, we can see the state diagram of socket in termination process bellow:



As we can see from the image above, in active close socket, because this socket will now know the status of the final step(ACK), so it needs delay in `TIME_WAIT` status in time(1 - 3 min). So in this time, the port is assigned to this socket cannot be used by any other sockets.

Moreover, if we did not have the TIME_WAIT state, consider a TCP connection between 12.106.32.254:1500 and 206.168.112.219:21. Suppose this connection is closed, and sometime later, a new connection is established using the same IP addresses and ports: 12.106.32.254:1500 and 206.168.112.219:21.

If either the client or server still has an old packet stuck in its buffer, that packet could arrive at the socket of the new connection. As a result, the new connection could receive a packet that actually belongs to the previous connection.

> Why `TIME_WAIT` is between 1 - 3 min?

In linux, we have the definition: Maximum segment lifetime(MSL). This time is the longest time that packet can be existed in buffer. So this state need at least 2xMSL (s) to avoid the problem above. We can see this time(2xMSL) by the command:

```
cat /proc/sys/net/ipv4/tcp_fin_timeout
```

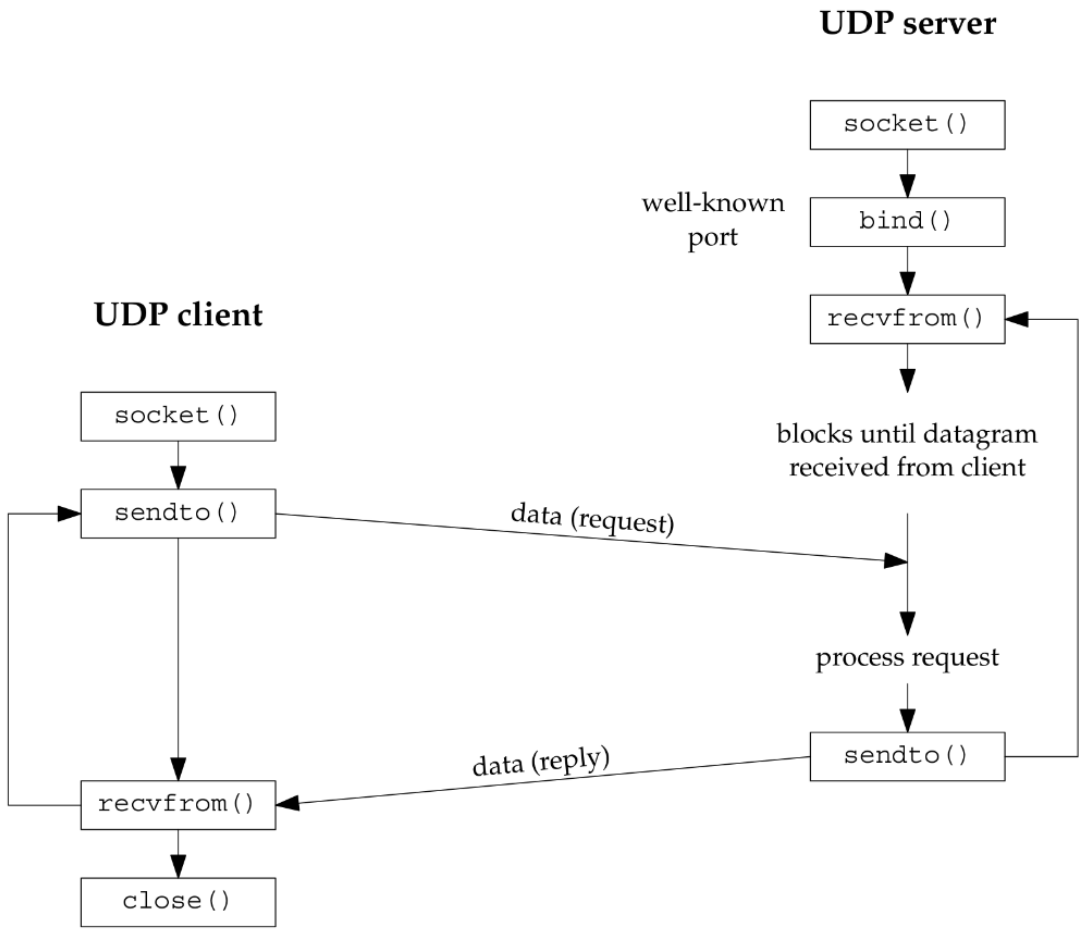
2.5. Some information of tcp socket program

- When a process **write** to a socket **without any read-fd processes**, the write-fd process is received signal `SIGPIPE`. The default action of this signal is terminating process. Therefore, we need write handle function to change this behaviour or ignore this signal.
- When server crash,after sending data to the tcp buffer in kernel of tcp client, the client will be blocked when try to read data from socket. tcp client will retransmiss this packet. After maximum retry time, client will give up and `read` return the `ETIMEOUT` error code
- If server reboots in retry time, server will lose all data from connection. Therefore, if server receive packets from clients, it will send RST packet to clients
- In the situation when device shutdown, the standard is send the signal `SIGTERM`, then from 5 to 20 seconds, init process will send the `SIGKILL` signal. This gives all running processes a short amount of time to clean up and terminate. Therefore we have sometime to catch and handle the `SIGTERM` signal, close all open socket fd.

3. UDP socket programming

UDP socket programming also have some function like TCP: `socket()`, `bind()`, `close()`. So I don't rewrite them in this section.

As we know, UDP is unreliable protocol, so we don't need any function to set up the connection between 2 processes like `connect()`, `listen()` and `accept()`. We can see an standard pattern bellow:



As we can see in the image above, UDP doesn't use `read` and `write` function because there is no connection between 2 socket, so we must use `recvfrom` and `sendto` function to temporary create a connection between 2 sockets.

```
ssize_t recvfrom(int sockfd, void *buff, size_t nbytes, int flags, struct sockaddr *from,
socklen_t *addrlen);
ssize_t sendto(int sockfd, const void *buff, size_t nbytes, int flags, const struct sockaddr
*to, socklen_t addrlen);
```

An example of tranfering data by using UDP socket programming:

```
/* server */
#define ADDRESS "10.0.0.77"
#define PORT 1234

int main()
```

```
{
    struct sockaddr_in seraddr;

    int sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    seraddr.sin_family = AF_INET;
    seraddr.sin_port = htons(PORT);
    inet_pton(AF_INET, ADDRESS, &seraddr.sin_addr);
    memset(&seraddr.sin_zero, 0, sizeof(seraddr.sin_zero));

    bind(sockfd, (struct sockaddr *)&seraddr, sizeof(seraddr));

    char temp[50];
    struct sockaddr_in clientaddr;
    socklen_t len = sizeof(clientaddr);

    int bytes = recvfrom(sockfd, temp, sizeof(temp) - 1, 0, (struct sockaddr *)&clientaddr,
&len);
    temp[bytes] = '\0';
    printf("%s\n", temp);
    char clientaddress[20];
    inet_ntop(AF_INET, &clientaddr.sin_addr, clientaddress, sizeof(clientaddress));
    int port = ntohs(clientaddr.sin_port);
    printf("receive from: IP = %s, port = %d\n", clientaddress, port);
    close(sockfd);
}

/* client */
#define ADDRESS "10.0.0.77"
#define PORT 1234

int main()
{
    struct sockaddr_in clientaddr;

    int sockfd = socket(AF_INET, SOCK_DGRAM, 0);

    char temp[50] = "khanh ne";
    struct sockaddr_in seraddr;
    seraddr.sin_family = AF_INET;
```

```
    seraddr.sin_port = htons(PORT);
    inet_pton(AF_INET, ADDRESS, &seraddr.sin_addr);
    memset(&seraddr.sin_zero, 0, sizeof(seraddr.sin_zero));
    socklen_t len = sizeof(seraddr);

    sendto(sockfd, temp, strlen(temp), 0, (struct sockaddr *)&seraddr, len);
    close(sockfd);
}
```

The result from server:

```
khanh ne
send from: IP = 10.0.0.77, port = 36985
```

3.1. Connected UDP socket

As we know, because UDP is **Connectionless**, so a client can send or receive from more processes in same time. If we want a client communicate with only one process in time, we can use `connect` function to create a temporary connection between 2 processes.

After using `connect`, client don't need to use `sendto` or `recvfrom` to send and receive data, it can use `send` and `recv` instead.

Some benefits when use connected UDP socket:

- Don't need call `sendto` and `recvfrom` with specific IP address and port each time when we need communicate.
- Receive from only one process which set a connection with this process by `connect` function.
- Connected UDP is more simple than Unconnected UDP. According to https://www.cs.columbia.edu/~lierranli/coms6998-10SDNFall2014/papers/netmap-ATC2012.pdf, Each packet when use sendto function need do more complex step to look for a route before sending, with connected UDP, we can reuse the route which save in route cache(kernel) to send.
- Asynchronous errors are returned to the process for connected UDP sockets. That mean, when we send data to invalid port or address, this process will be received the error alert packet, but `sendto` cannot catch it. With `send`, we can fix this problem.

> We can also call more than `connect` function in 1 process to connect with more than 1 IP/Port because UDP is not connection like TCP, we can also maek the

> socket to unconnected by the flag: `AF_UNSPEC`.

3.2. Lack of flow control with UDP

As we know, because UDP don't have the way to check the sequence number of receiving packet, so when we send many packets at time, the buffer of receive socket can be full and it can miss packet from sender.

For example, when sender send 2000 datagram fast:

```
for (int i = 0; i < 2000; i++)
    {
        sendto(sockfd, temp, strlen(temp), 0, (struct sockaddr *)&seraddr, len);
    }
```

The result of receive socket when we count the number of datagram:

```
receive total : 1525 datagrams
```

We can use the `setsockopt`(in the later section) to reduce this problem.

4. I/O multiplexing

In a process, there are more than 1 input at the same time: From standard input(STDIN) and Socket like this example:

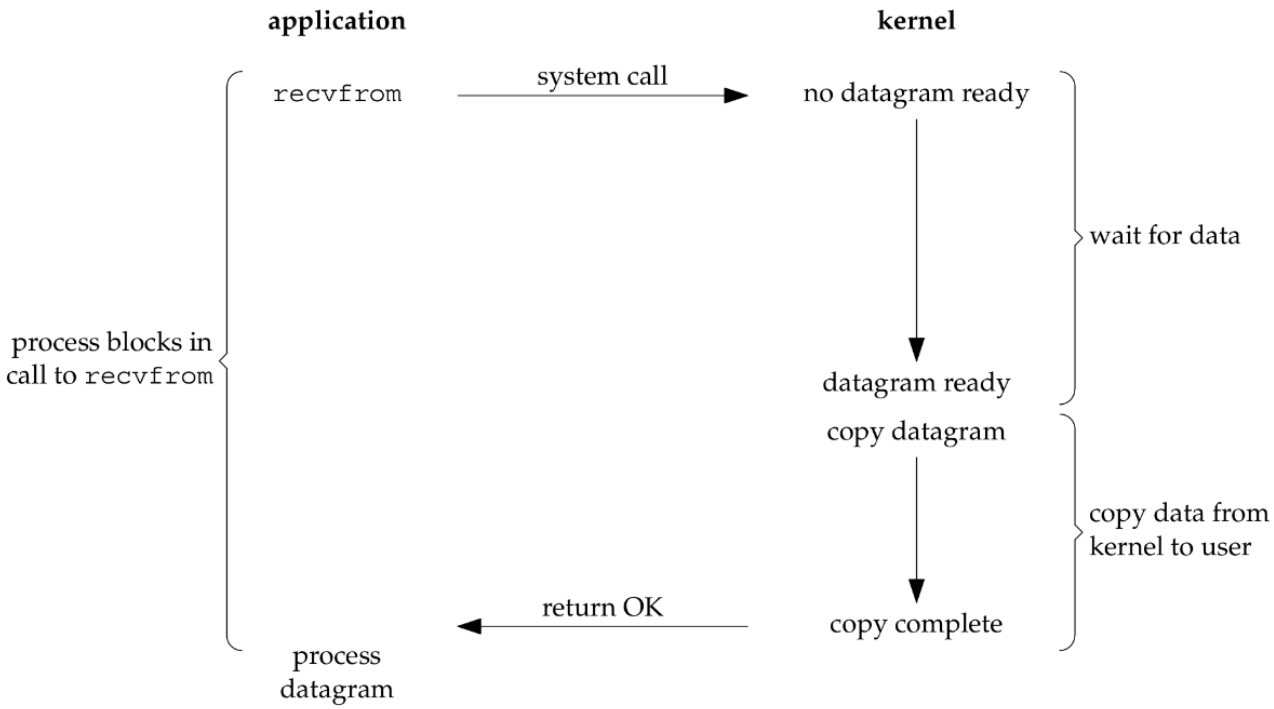
```
while (1)
    {
        char result[50];
        memset(&result, 0, sizeof(result));
        getchar();
        int bytes = read(clientfd, result, sizeof(result) - 1);
        if (bytes > 0)
        {
            result[bytes] = '\0';
        }
        printf("client to server: %s", result);
    }
```

The problem is when the process is blocked by `getchar()`, the server process is killed, then the client process will never see the EOF until it read from the socket. We can solve this problem by the technique: I/O multiplexing

Before all, we need to know the definitions:

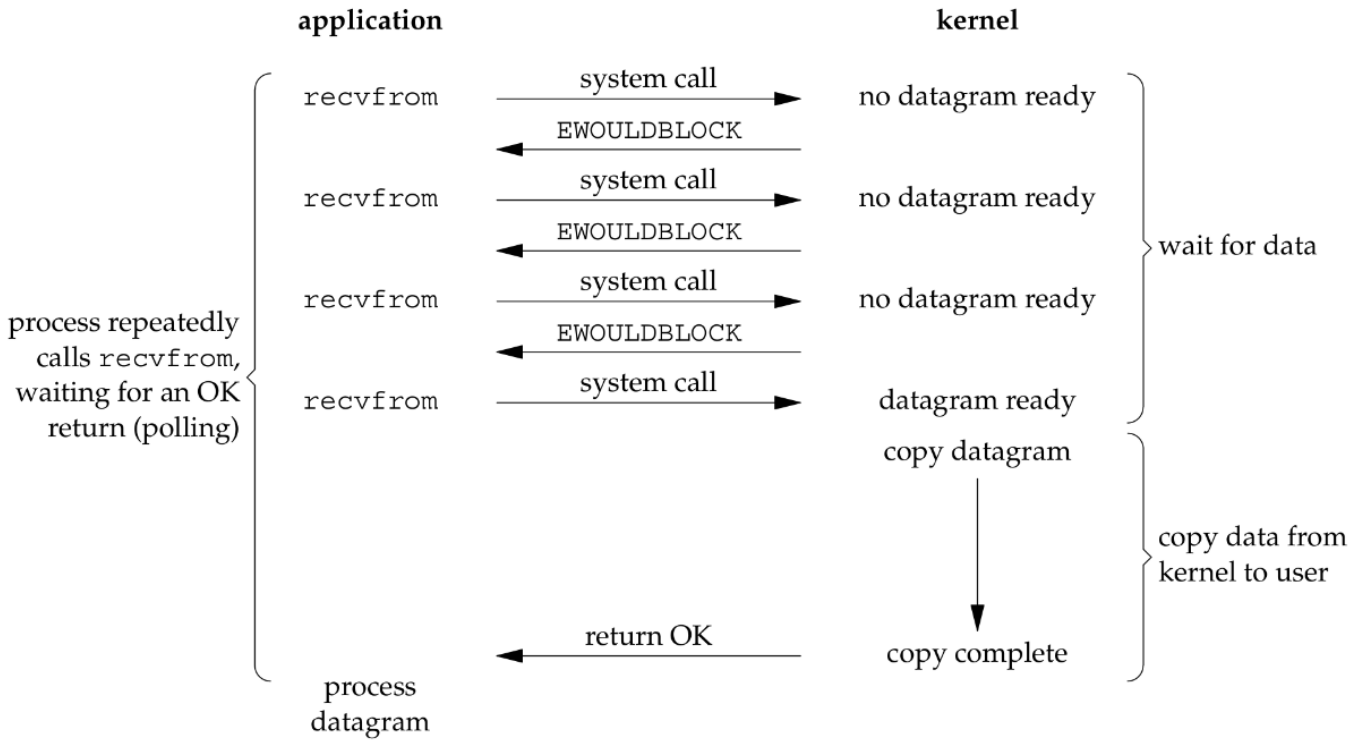
Blocking IO model:

This model make process block until receive data from the input which make this process blocked:



Non-Blocking IO model:

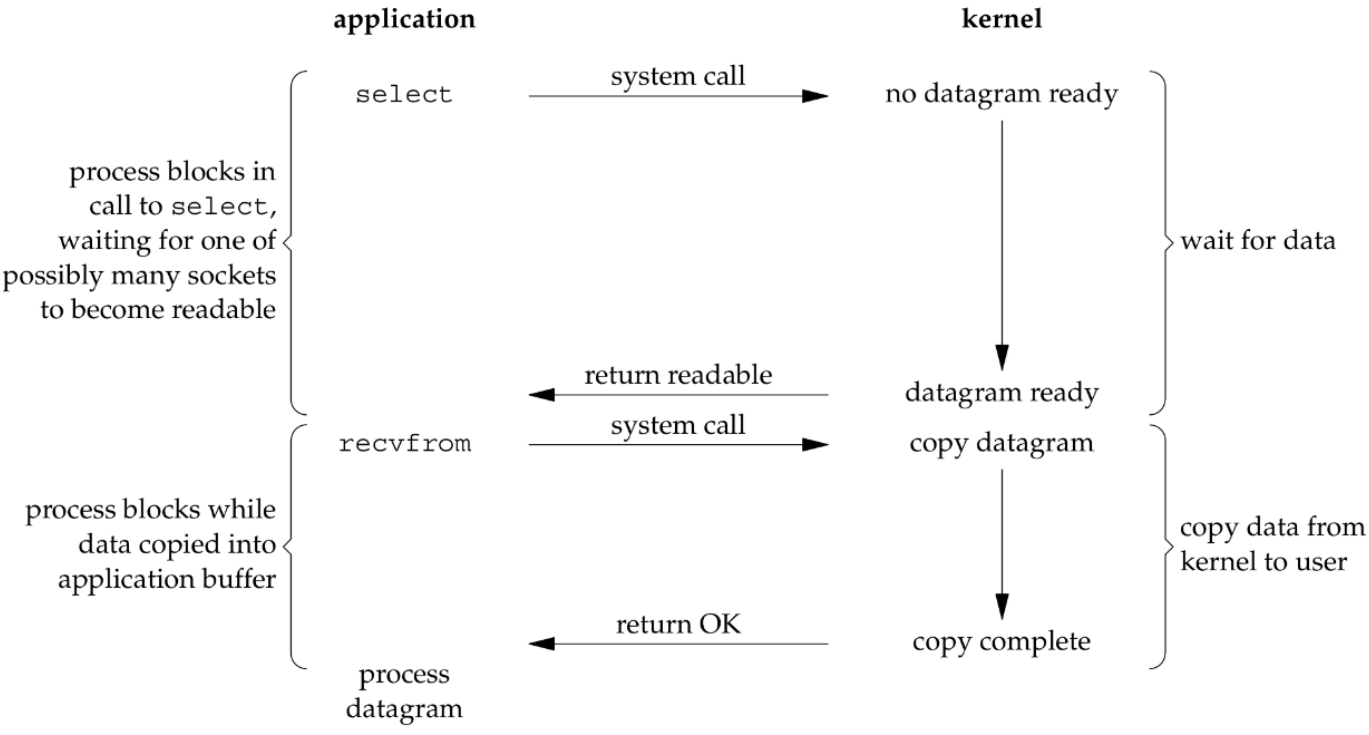
In this model, instead of blocking in a input, the input function will return immediatly with the error code. We can build this model by the while loop to check everytime when the IO ready.



IO Multiplexing model

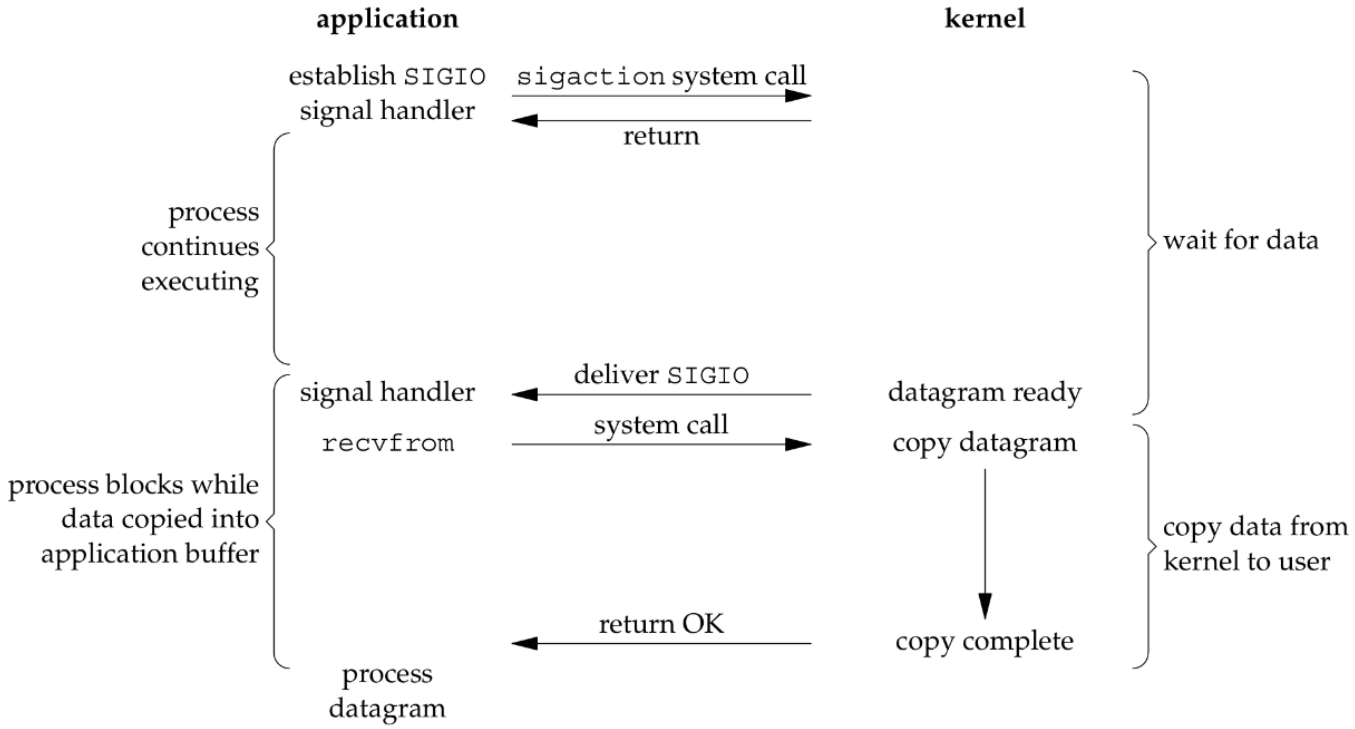
This model is same the first model because both of them make the process block. However, the IO multiplexing model make the process blocked in `select()` sytemcall.

Therefore, the process will be blocked by select and will wake up until 1 of the input fd readable instead of blocking until the fisrt input readable.



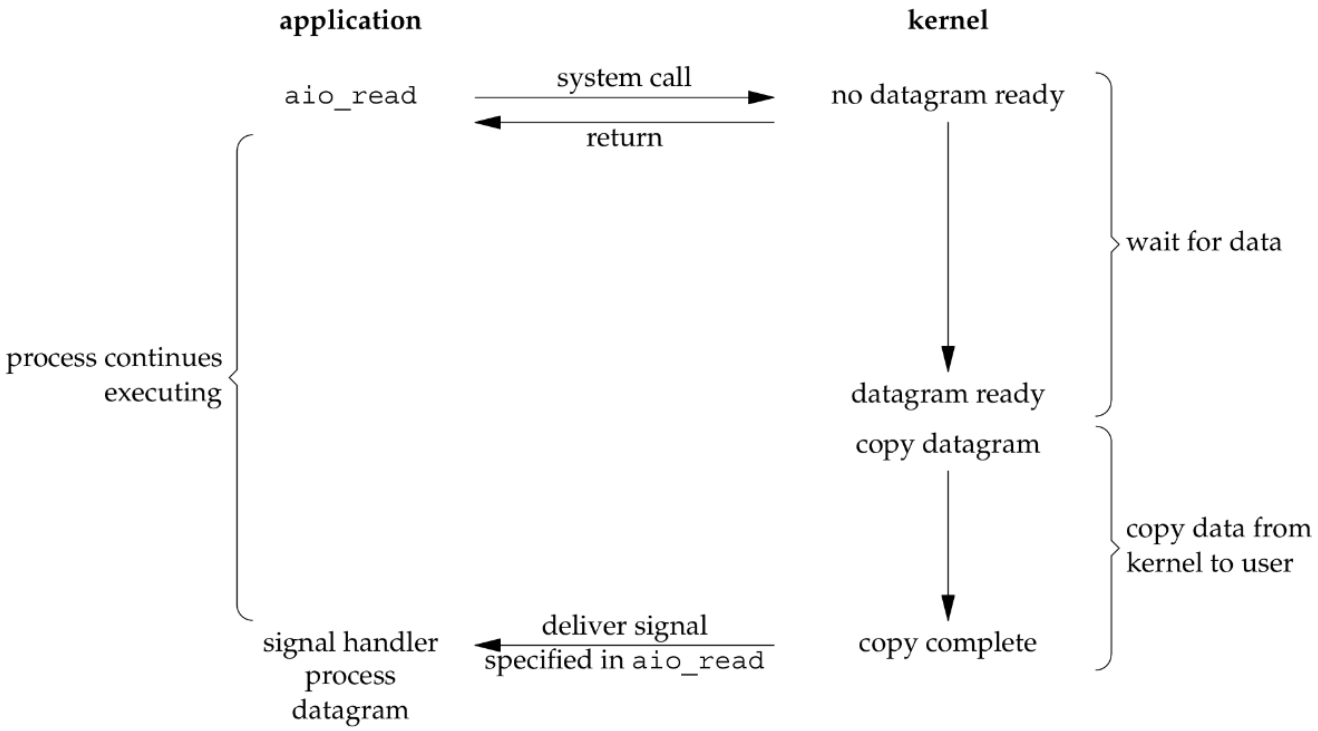
Signal-Driven I/O Model

We can also use signal to notify the process when the input is readble:

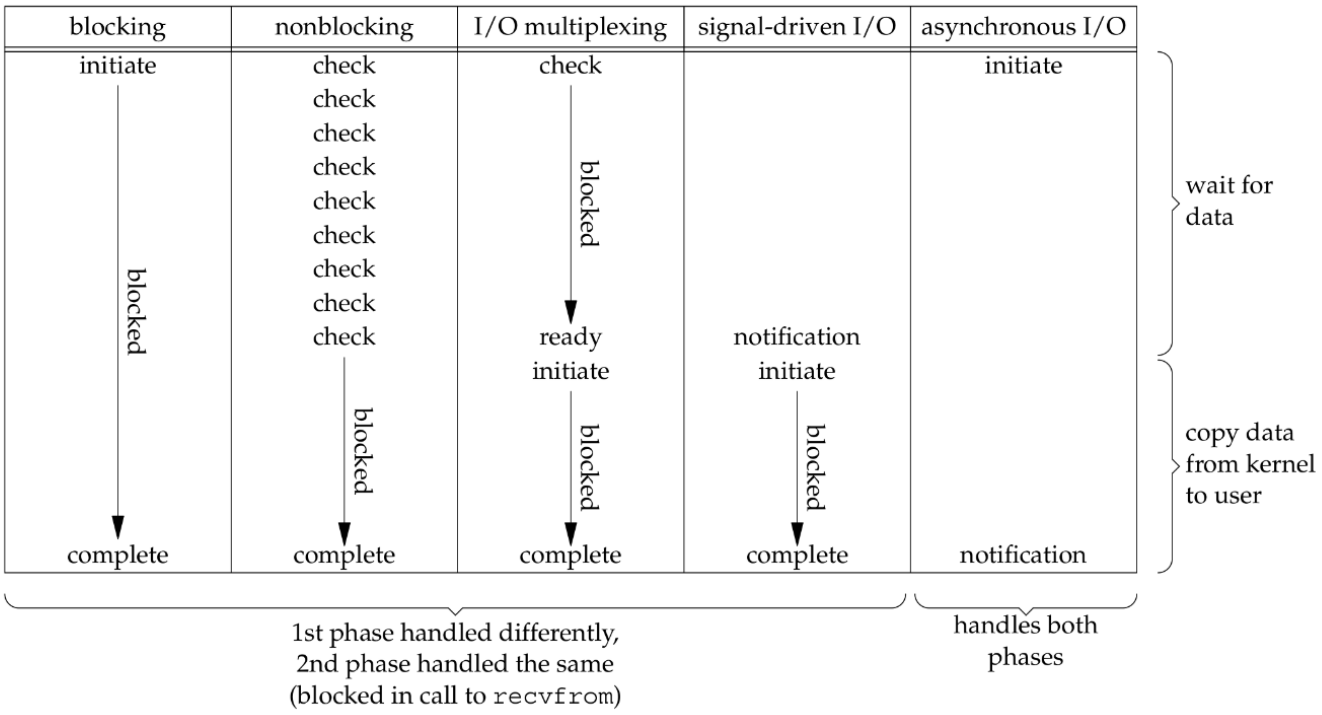


Asynchronous I/O Model

In the previous model, although processes don't need to wait data in tranfering data step, but processes must call systemcall to get data from kernel to userspace. Therefore, by using Asynchronous I/O Model, the kernel will do all 2 step, that mean the process will receive the signal when the data is stored in userspace instead of kernel space.



> We can see the comparision of 4 method in the image bellow:



4.1. Select function

`Select` is the system call which help process block until an input ready.

```
int select(int maxfdp1, fd_set *readset, fd_set *writeset, fd_set *exceptset, const struct timeval *timeout);
```

> When we use `select`, after each time when process wake up and select return, the `fd_set` elements will be set to 0, so we must set them to the meaningful value again.

The timeout element value situation:

- NULL: Wait forever until an I/O ready
- non-interger number: wait until the number time
- 0: Check only 1 time, return immediately(Polling situation)

> The maximum number of fd in fd_set is 1024. We can pass larger number of fd to this pointer, but in kernel, this value is unchanged. So kernel will only read from 0 to 1024, all value with fd > 1024 will be drop.

We can fix this problem by the `poll` systemcall:

4.2. Poll function

`Poll` is similar purpose with `select`, but with `poll`, we can pass more than 1024 fd to check by using array of struct with no limit fd:

```
int poll(struct pollfd *fdarray, unsigned long nfds, int timeout);
```

Now we have a problem, when the process is blocked by `select`, this process will receive all signal, so the interupt can be occured. To avoid this error, we can use the `pselect` function:

```
int pselect(int maxfdp1, fd_set *readset, fd_set *writeset, fd_set *exceptset, const struct timespec *timeout, const sigset_t *sigmask);
```

When we pass any signal to sigset_t, this signal will mark pending, after returning of pselect and the process wake up, this process will be received signal from pending.

However, with all system calls above, the problem is the complexity of checking status of all fd. With select, pselect and poll, this complexity is O(N). That mean although there is only 1 I/O ready, but kernel always check every I/O.

Moreover, this system calls have some problems with data struct of fd.

- With select, we must initialize the data structure before each call because the kernel will change this structure value.
- With poll, because the limitation of the fd number, when this number become to large, the step copy from user to kernel and from kernel to user when returning waste time.

Finally, this functions tell process how many fd number is ready when return instead of exactly ready fd.

To solve this problems, we can use the advanced functions: `epoll`

4.3. Epoll function

Before talking about epoll, we must understand the definition:

- **Level-triggered notification**: A file descriptor is considered to be ready if it is possible to perform an I/O system call without blocking.
- **Edge-triggered notification**: Notification is provided if there is I/O activity (e.g., new input) on a file descriptor since it was last monitored.

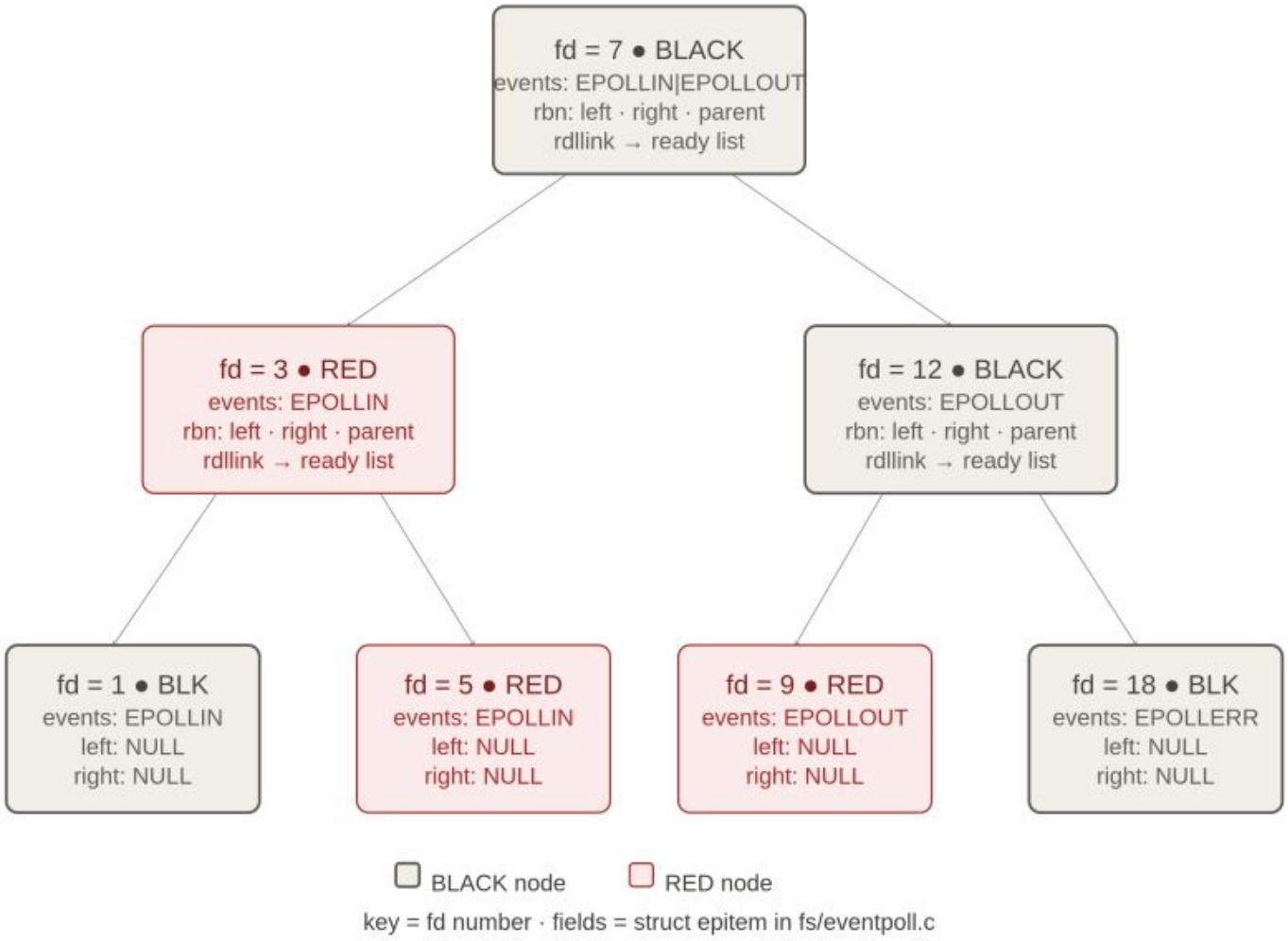
Now let's begin to `epoll`:

Not like the other method, epoll have 2 data structure to manage:

- **Interest list**: this list include all fd which need monitor
- **ready list**: this list include all fd which I/O is ready

4.3.1. Interest list

Interest list use red-black tree data structure(DS) to manage. This DS help kernel can insert, remove and search step in O(logN) time.



4.3.2. Ready list

Ready list use doubly linked list to present. Each I/O have a callback function will be executed at time when I/O ready. This callback will push the fd to ready list in O(1) time.

After that, whenever we get ready fd by epoll, we just get from ready list without checking again in all fd like others

4.3.3. Epoll API

- To create a epoll instance:

```
int epoll_create(int size);
```

- Add, modify, delete items from interest list:

```
int epoll_ctl(int epfd, int op, int fd, struct epoll_event *ev);
```

> We can see the maximum number of fd in epoll by the command: `cat /proc/sys/fs/epoll/max_user_watches`

- Block and wait until at least 1 I/O ready:

```
int epoll_wait(int epfd, struct epoll_event *evlist, int maxevents, int timeout);
```

Compare poll and epoll in 10,100,1000 and 2000 fd, each time get the average number of 2000 times:

	10	100	1000	2000
poll(us)	1.440	5.953	58.278	113.877
epoll(us)	0.331	0.352	0.335	0.323
compare	4.35x	16.92x	174.10x	352.39x

You can get the source code in https://github.com/220222112-PhamNgocKhanh/Zhone_trainee/tree/main/socket_programming/compare_poll_epoll

> I will list all situation when I/O ready:

Readble:

- **Socket Receive Buffer**: The number of byte inside socket $\ge$ Low-Water Mark(1 byte)
- **Peer Closed Connection**: The peer application shutdown the write
- **Listening Socket**: At least 1 TCP connection is completed
- **Pending Socket Error**: An error in socket(ex: RST packet), call read will return -1 and error code
- Other special fd(File, Pipe, Timer, Event):

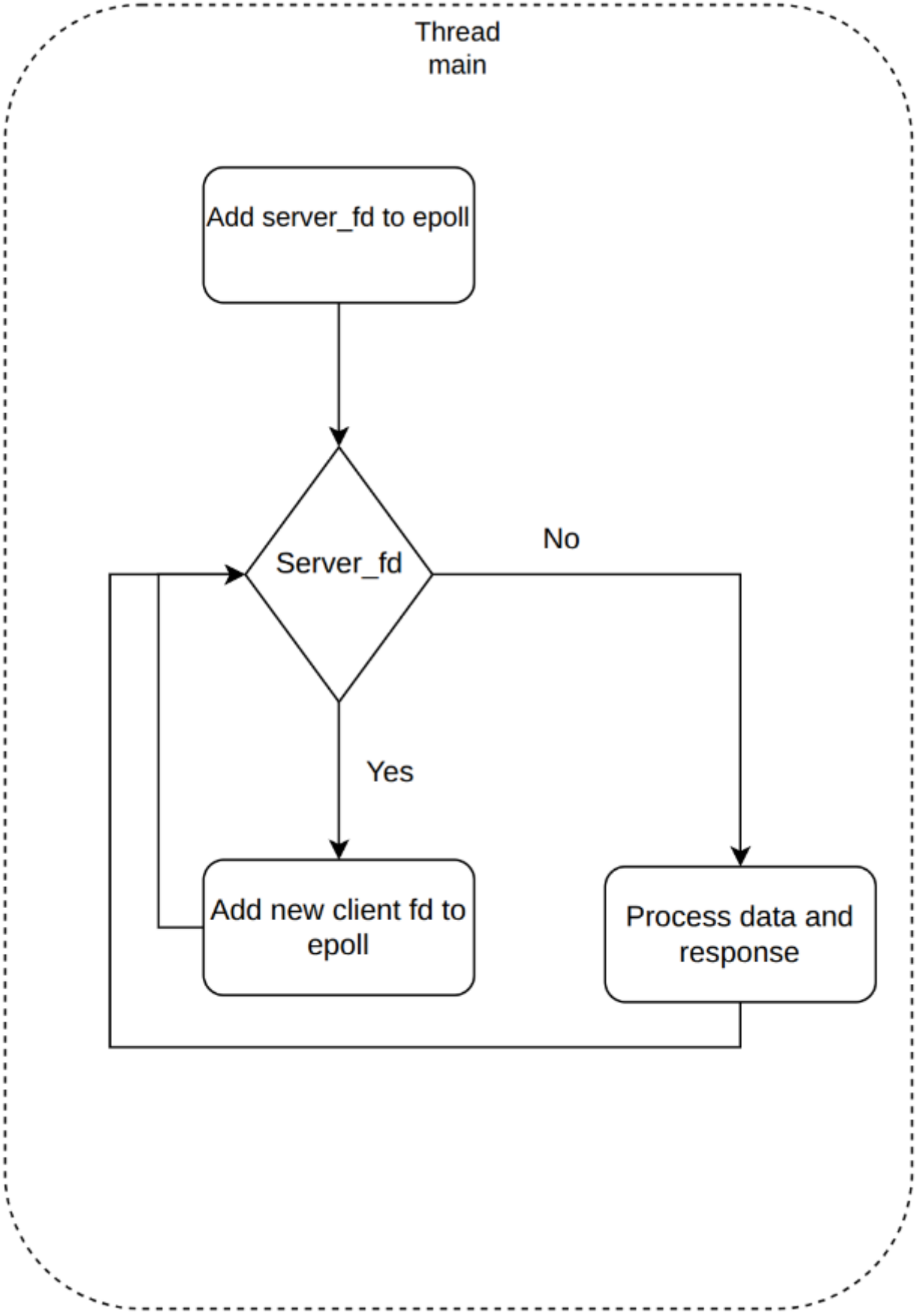
Writeable:

- **Socket Send Buffer**: The available space in the socket send buffer is $\ge$ Low-Water Mark
- **Asynchronous Connection Completed**: A non-blocking connect() previously returned EINPROGRESS. When the 3-way TCP handshake completes successfully, the socket becomes writable.
- **Pipe Read-End Closed**: The socket with the peer application call `shutdown_RD`
- **Pending Socket Error**: An error of socket

Other exceptions

4.4. Single and multi-thread with I/O multiplexing

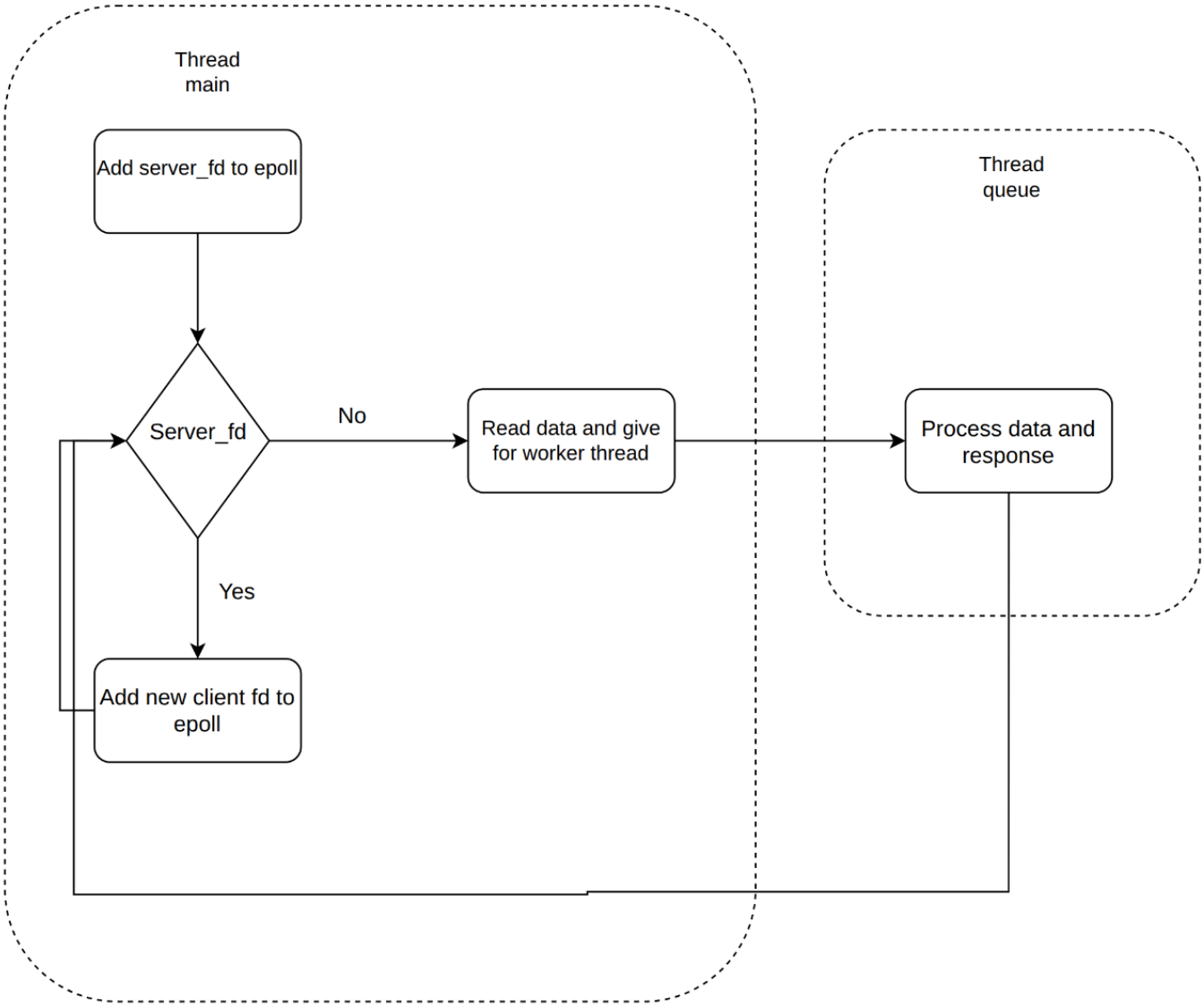
The first situation is single thread with I/O multiplexing.



Imagine when the step **process data and response** run in long time, the process will not do anything, so any request from other clients will be delayed. Besides, this pattern doesn't use the

power of multicore in CPU.

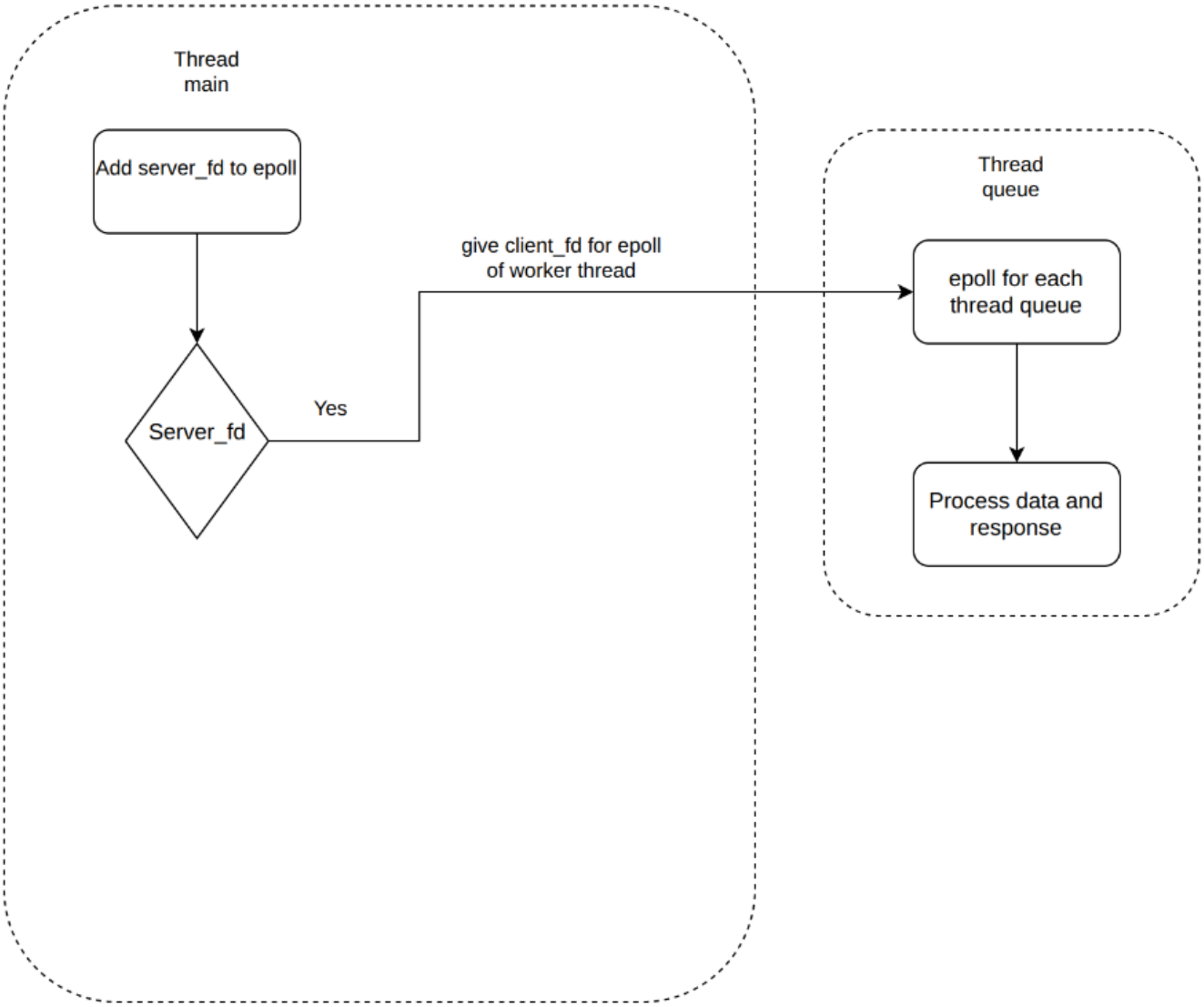
To solve this problem, we can use the next pattern: **single epoll and thread pool** like this:



This pattern have a problem:

- We must use mutual exclusion to protect thread queue
- Main thread need to read data from kernel of all I/O, don't really use the multi-core power

To solve all problem above, we can use the third pattern: multi epoll and thread pool.



Revision #6
Created 2026-09-23 04:37:10 UTC by Khanh Pham
Updated 2026-10-14 07:24:42 UTC by Khanh Pham